



UNIVERSITÀ DI PISA

MASTER OF SCIENCE IN COMPUTER ENGINEERING
LARGE-SCALE AND MULTI-STRUCTURED
DATABASES

BookSphere

Books Review Social Network

Authors:

Danilo PARISI

Matteo IORI

Matteo BILLI CIANI

A.Y. 2025–2026

Contents

1	Introduction	2
1.1	Project Overview	2
1.2	Motivation and Key Features	2
2	System Design	4
2.1	System Architecture	4
2.1.1	Architectural Layers	4
2.2	Requirements Analysis	5
2.2.1	Main Actors	5
2.2.2	Functional Requirements	5
2.2.3	Non-Functional Requirements	8
2.3	Expected Workload Distribution	8
2.3.1	Workload Profile (OLTP vs. OLAP)	8
2.4	UML Class Diagram	9
2.5	User Interface Concept and Mock-ups	9
2.5.1	Unregistered User Experience	9
2.5.2	Registered User Experience	10
2.5.3	Administrator Experience	11
3	Data Modeling	12
3.1	Data Source and Volume	12
3.1.1	ETL Pipeline	12
3.1.2	Deployment Scripts	13
3.2	Document Store: MongoDB	14
3.2.1	Collections Overview	14
3.2.2	Schema Design (JSON)	14
3.3	Graph Store: Neo4j	18

3.3.1	Node Types	18
3.3.2	Relationship Types	18
3.3.3	Graph Schema Visualization	19
3.3.4	Design Rationale	19
3.3.5	Relationship Modeling Note	20
3.4	Polyglot Persistence Strategy	20
3.4.1	Synchronization Matrix	20
3.4.2	Error Handling Strategy	22
4	Implementation	23
4.1	Technologies and Frameworks	23
4.2	Directory Structure and Modules	23
4.2.1	Config	23
4.2.2	Controllers	24
4.2.3	DTO	26
4.2.4	Mappers	26
4.2.5	Model	26
4.2.6	Repositories	26
4.2.7	Services and Async	27
4.2.8	Utils	28
4.2.9	Validation	28
4.2.10	Exceptions	29
4.3	MongoDB Relevant queries	29
4.3.1	Ranking	29
4.3.2	Revaluation	31
4.3.3	Yearly Wrapped	32
4.4	Neo4j Relevant Queries	33
4.4.1	User Recommendations	34
4.4.2	Internationality Index	34
4.4.3	Genre Influencer	35
4.5	API Documentation	36
4.5.1	API Architecture	36
4.5.2	Authentication Mechanism	36
4.5.3	Key Endpoints Overview	37
4.5.4	API Examples	39

4.5.5	Error Handling	41
4.5.6	Rate Limiting and Security	41
5	Databases Choices and Distributed Architecture	42
5.1	Polyglot Persistence Strategy	42
5.2	Indexes Justification and Validation	42
5.2.1	MongoDB Indexes	42
5.2.2	Neo4j Indexes	44
5.3	Virtual Machine Organization and Replica Sets	44
5.3.1	Document Database Deployment	44
5.3.2	Graph Database Deployment	45
5.4	Sharding Strategy	45
5.4.1	Shard Key Selection Rationale	45
5.4.2	Chunk Size Configuration	47
5.4.3	Future Considerations	47
5.5	CAP Theorem Considerations	47
5.6	Inter-Database Consistency	47
5.6.1	Eventual Consistency via @Async	48
5.6.2	Resilience with @Retryable	48
6	System Testing and Performance Evaluation	49
6.1	Introduction	49
6.2	Java-based Testing Strategy	49
6.2.1	Testing Approach	49
6.2.2	Test Configuration	50
6.2.3	Test Coverage	51
6.2.4	Key Consistency Patterns Verified	51
6.3	Performance Testing with Gatling	51
6.3.1	Testing Methodology	52
6.3.2	BasicLoadTest - Baseline Performance	52
6.3.3	StressTest - Breaking Point Analysis	53
6.3.4	SpikeTest - Sudden Traffic Surge Recovery	54
6.3.5	EnduranceTest - Long-Duration Stability	54
6.3.6	DatabaseIntensiveTest - Analytics Query Performance	55
6.3.7	Conclusion	56

6.4	Conclusion	56
6.5	API Testing with Postman Collection	56
6.5.1	Collection Overview: "BookSphere Enhanced User Journey" . . .	56
6.5.2	Key Testing Features	57
6.5.3	Storytelling Scenario: "The Social Impact"	58
6.5.4	Example: Cross-Database Consistency Test	58
6.5.5	Newman CLI Execution	58

Chapter 1

Introduction

Welcome to **BookSphere**, the ultimate social platform for book lovers designed to help you organize your reading life and connect with a global community of readers like you.

1.1 Project Overview

Imagine a place where your personal library is always at your fingertips, curated exactly how you live it—sorting your journey from the books you dream of reading, to the ones currently on your nightstand, to the stories that have already shaped you.

But a library should not be a static place. BookSphere allows you to build a circle of literary companions by following friends and fellow readers. Your feed transforms into a living book club, where you can track what your network is devouring in real-time, share reactions, and draw inspiration from the shelves of those you know best.

1.2 Motivation and Key Features

Beyond your personal circle, the motivation behind BookSphere is to connect users to a pulse of like-minded individuals, filtering out the noise of generic popularity. The platform introduces unique features designed to reward quality engagement:

- **Real Influencers:** We surface voices trusted not for their volume, but for the quality of their insight identifies, consistency and knowledge of a genre. Expert reviewers identified by the quality of their engagement rather than just follower count.
- **Trending Probability:** Through advanced data analysis, we help you discover the next literary phenomenon before it hits the mainstream charts.
- **Internationality Index:** We visualize how a story travels across borders, showing you that the book in your hands is being turned by another reader in Tokyo, New York, or Rome at this very moment.

- **Yearly Wrapped:** Ultimately, BookSphere is about your story. As you interact and rate, the platform aggregates your history to generate a personalized statistical summary at the end of the year—visualizing your top authors, favorite genres, and reading milestones.

Chapter 2

System Design

This chapter outlines the architectural blueprint of **BookSphere**. We begin by describing the high-level system design and technology stack, followed by a detailed analysis of functional and non-functional requirements, and concluding with the visual interface design.

2.1 System Architecture

The BookSphere platform is built upon a robust **3-Tier Architecture**, designed to ensure separation of concerns, scalability, and maintainability.

2.1.1 Architectural Layers

- **Presentation Layer (Frontend):** The client-side application is designed to be responsive and intuitive. It communicates with the backend via RESTful APIs, handling user interactions and data visualization (e.g., rendering the social graph or library shelves).
- **Logic Layer (Backend):** The core of the system is developed using **Java** with the **Spring Boot** framework. This layer handles business logic, security (Authentication/Authorization), and acts as an orchestrator between the two different databases. We utilize **Lombok** to reduce boilerplate code and ensure cleaner class definitions.
- **Data Layer (Polyglot Persistence):** The system adopts a hybrid storage strategy:
 - **MongoDB:** Acts as the primary document store for Books, User Profiles, and detailed Reviews.
 - **Neo4j:** Serves as the high-performance graph engine to manage social connections (*FOLLOWS*) and interaction patterns (*READS*, *LIKED*) for the recommendation algorithms.

2.2 Requirements Analysis

2.2.1 Main Actors

The system serves three main categories of users, each with distinct privileges and interaction capabilities:

- **Administrator:** The super-user responsible for platform management. Their tasks range from maintaining the consistency of the catalog (adding/updating books and authors) to community moderation (deleting content and banning users).
- **Registered User:** The primary target audience of the platform. These users can fully interact with the system by creating social connections, writing reviews, curating their library, and generating personal analytics.
- **Unregistered User (Guest):** Users who access the platform in *"read-only"* mode. They can explore the catalog, view public rankings and reviews, but cannot perform active interactions (rating, following, posting).

2.2.2 Functional Requirements

The functional requirements define the specific behaviors and functions of the system, categorized by the actor scope.

General & System Requirements

These requirements apply to the core logic of the platform and are available to all users (including Guests) or performed automatically by the system background processes.

Search & Discovery

1. The System must permit users to search for books and authors using different filters (genre, year, name).
2. The System must allow users to search for other profiles and view their reviews.
3. The System must enable any user to read reviews associated with a book.

System Analytics & Computation

4. The System must compute, for every newly added book, a probability score indicating the likelihood of the book going viral.
5. The System must compute a metric representing an author's versatility based on the variety of genres covered in their works.

6. The System must compute a "How Far a Book Travels" index, measuring the geospatial distribution of a book's readers across the globe.
7. The System must generate various rankings, such as top books or authors, filtered by year or genre.
8. The System must calculate and display the average rating for authors and books for every year.
9. The System must identify trending books based on the volume of reviews and ratings in the current month.
10. The System must automatically identify "Influencers" within the application based on engagement quality metrics.

Access

11. The System must enable an Unregistered User to create a new account (Sign-up).

Registered User Requirements

These features are exclusive to authenticated users.

Personalization & Insights

1. The System must provide book recommendations based on the user's interaction history (favorite genres, authors) and global trends.
2. The System must generate a "Wrapped" style summary of the user's reading year (statistics, top genres).
3. The System must enable users to view a complete list of publications for a specific author.
4. The System must allow users to retrieve all reviews for a specific book on demand.

Content Creation & Library Management

5. The System must allow a user to review books, providing a numerical evaluation (score) and an optional written comment.
6. The System must allow each user to assign a status to a book, selecting from: *To-Read*, *Reading*, or *Read*.
7. The System must enable users to create and manage custom lists of books.

Social Interaction

8. The System must enable users to follow other profiles and view their latest activities in a dedicated feed.
9. The System must enable users to "like" or "unlike" specific books.
10. The System must enable users to "like" or "unlike" reviews written by others.
11. The System must allow users to view their list of friends/followed accounts.

Account Management

12. The System must permit users to Log In and Log Out securely.
13. The System must allow users to delete their account.

Administrator Requirements

Privileged operations for content and user management.

Catalog Management

1. The System must enable an Admin to add new books, authors, or genres to the database.
2. The System must enable an Admin to update existing information related to books, authors, or genres.
3. The System must enable an Admin to remove a specific book, author, or genre.

Moderation & Oversight

4. The System must allow an Admin to view the profile and data of any Registered User.
5. The System must enable an Admin to view any review posted on the platform.
6. The System must enable an Admin to delete reviews deemed inappropriate.
7. The System must enable an Admin to ban a Registered User¹.

¹Banning prohibits access but preserves specific user metadata to prevent re-registration with the same credentials.

2.2.3 Non-Functional Requirements

These requirements define the quality attributes of the system.

1. The System must follow RESTful design principles for API communication.
2. The System must implement backup and redundancy strategies to avoid permanent data loss.
3. The System must encrypt sensitive user data, specifically passwords, using industry-standard hashing algorithms.
4. The System must be highly available and fault-tolerant to ensure continuous service.
5. The System must enforce an Eventual Consistency model between the Document Database (MongoDB) and the Graph Database (Neo4j) to optimize performance over strict synchronization.

2.3 Expected Workload Distribution

A critical aspect considered when designing the system and database architecture is the expected workload profile. Given the social nature of the application, BookSphere is designed to handle a **Read-Heavy** workload, typical of large-scale social platforms.

2.3.1 Workload Profile (OLTP vs. OLAP)

The system operates primarily as an **OLTP (Online Transaction Processing)** environment, characterized by a high volume of short, atomic transactions (e.g., posting a review, following a user). However, features such as *Yearly Wrapped* and *Trending Probability* introduce periodic **OLAP (Online Analytical Processing)** workloads, requiring the aggregation of vast datasets.

The operational split is estimated as follows:

- **Read Operations (~80-85%):** The majority of traffic consists of content consumption: browsing book details, traversing the social graph (viewing friends' activities), and authentication.
 - *Architectural Implication:* This imbalance justifies the use of **Replica Sets** in MongoDB. By setting the read preference to `secondaryPreferred`, we offload the heavy read traffic to secondary nodes, keeping the Primary free for write operations.
- **Write Operations (~15-20%):** Writes are further categorized based on frequency and complexity:
 1. **High-Frequency Writes (~10-15%):** Lightweight interactions such as 'Likes' and 'Follows'. These require low latency and can tolerate *Eventual Consistency*.

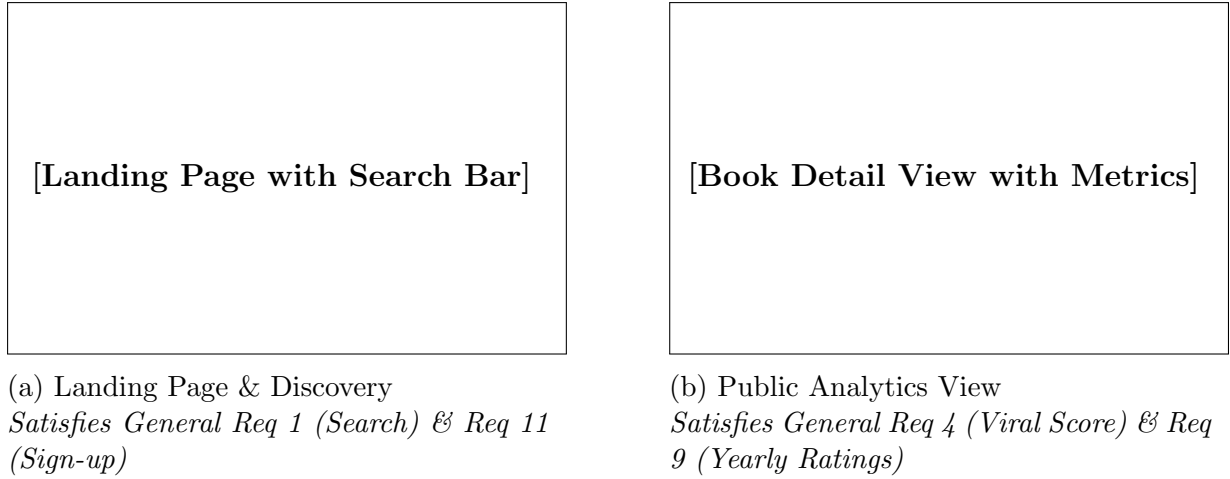


Figure 2.2: Guest interface focusing on discovery and public data visualization.

2.5.2 Registered User Experience

The Authenticated User interface enables **Personalization**, **Content Creation**, and **Social Interaction**. The designs below highlight the transition from passive reading to active engagement.



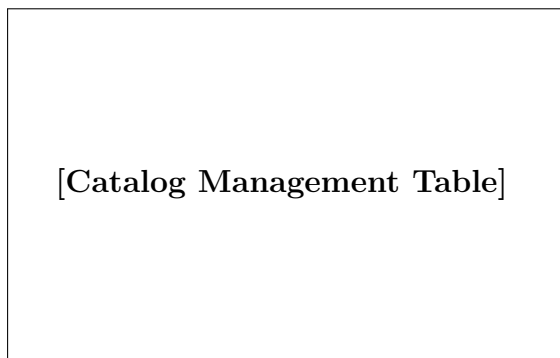
Figure 2.3: Core interaction screens for Registered Users.



Figure 2.4: Personalized Analytics Dashboard.
Satisfies Reg-Req 2 ("Wrapped" Summary)

2.5.3 Administrator Experience

The Admin interface prioritizes **Catalog Management** and **Moderation**. These views are data-dense to facilitate efficient system oversight.



(a) Database Management
Satisfies Admin Req 1-3 (Add/Update/Remove)



(b) Moderation & Banning
Satisfies Admin Req 7 (Ban User) & Req 6 (Delete Review)

Figure 2.5: Administrative panels for maintaining system integrity.

Chapter 3

Data Modeling

This chapter details the data persistence layer of **BookSphere**. Given the polyglot nature of the architecture, the data modeling strategy is split between a **Document-Oriented** approach (*MongoDB*) for content management and analytics, and a **Graph-Oriented** approach (*Neo4j*) for social interactions and recommendation algorithms.

3.1 Data Source and Volume

To ensure the system was tested against realistic data volumes and complex interaction patterns, we implemented a hybrid data generation strategy using custom automation scripts. The core catalog and foundational rating data are sourced from the **Amazon Books Reviews Dataset** (providing real-world book metadata and user ratings).

This real data is a user base of approximately 10,000 active accounts. The generation process automatically populates the polyglot persistence layer:

- **Amazon Books Reviews Dataset (~5GB):** Provides the core catalog of books and real-world reviews spanning from 1996 to 2014.
- **Book-Crossing Dataset:** Used as a supplementary source for user ratings to increase data *variety* and *volume*.

3.1.1 ETL Pipeline

The dataset generation follows a 6-step ETL (Extract, Transform, Load) pipeline implemented in two Python scripts:

1. **Books Merge:** Amazon `books_data.csv` and BookCrossing `Books.csv` are merged by title match. Configuration limits (`MAX_BOOKS=20,000`, `MAX_REVIEWS_PER_BOOK=50`, `MAX_USERS=12,000`) control the final dataset size.
2. **Users Generation:** Users are sampled from both sources, enriched with BCrypt-hashed passwords, randomized emails, and ISO country codes from 50 countries via

MongoDB (Document Store)	Neo4j (Graph Store - CSVs)
• 37.1MB for reviews • 27.7MB for books • 12.4MB for users • 3.5MB for authors	• 4.32MB for Nodes (User, Book, Review...) • 24.9MB for Relationships (LIKES, FOLLOWS...)

Table 3.1: Database Volumes by Collection.

the
texttttrandomuser API pattern.

3. **Reviews Processing:** BookCrossing ratings (0–10 scale) and Amazon ratings (1–5 scale) are both normalized to the unified 10–100 scale used by BookSphere. Source provenance is preserved in the `source` field.
4. **Data Enrichment:** The pipeline injects computed fields: ISBNs are cross-matched, bookshelves are populated (`texttttread + texttttwant_to_read`), and pre-aggregated statistics are calculated (`stats_per_year`, `month_score`, `popular_reviews_snapshot`, `author ratings_count/sum_ratings`).
5. **MongoDB Export:** 4 JSONL files (`books.jsonl`, `users.jsonl`, `reviews.jsonl`, `authors.jsonl`) are generated for MongoDB ingestion via `mongoimport`.
6. **Neo4j Export:** A second script reads the MongoDB JSONL output and generates 14 CSV files (5 node types + 9 relationship types) for Neo4j’s `LOAD CSV` bulk import. Relationship inference is *smart*: genre likes are derived from the user’s actual reading history (top 3 genres), author likes from their bookshelf preferences, and book likes from high ratings (≥ 80).

3.1.2 Deployment Scripts

Three shell scripts automate database population:

- `populate_mongo.sh`: Single-node MongoDB import with index creation.
- `populate_local_cluster_mongo.sh`: 3-node replica set import (`localhost:27017`, `27018`, `27019`).
- `populate_neo4j.sh`: Copies CSVs to the import directory, clears the database, creates indexes on `mongoId/name`, and executes `LOAD CSV` for all nodes and relationships.

Data Variety: By merging Amazon and BookCrossing, the dataset satisfies the multi-source requirement. *Velocity:* Old reviews (pre-2024) are excluded from trending and monthly analytics, satisfying the temporal relevance requirement.

3.2 Document Store: MongoDB

MongoDB acts as the primary "Source of Truth" for the system. It stores the complete information about books, users, and reviews. The schema design prioritizes **Read Performance** through specific NoSQL patterns.

3.2.1 Collections Overview

The database `booksphere.db` contains the following main collections:

1. **books:** The central catalog entity.
2. **users:** Stores authentication data and the user's library.
3. **reviews:** Stores the full text of reviews. Linked to books and users via references.
4. **authors:** Stores aggregated statistics about authors (e.g., total ratings).

3.2.2 Schema Design (JSON)

Books Collection

We implemented the **Bucket Pattern** (`stats_per_year`) to make historical ranking queries instant, avoiding the need to scan millions of review documents. The **Snapshot Pattern** (`recent_reviews_snapshot`) allows the frontend to render the book details page with the latest reviews in a single query.

```
1 {
2   "_id": "698db76b4a1f5c8fa49331a8",
3   "title": "The Fellowship of the Ring",
4   "publication_year": 2000,
5   "description": "Begin your journey into Middle-earth... The
6     inspiration for the upcoming...",
7   "availability": "ACTIVE",
8   "source": "amazon_master",
9
10  "author": {
11    "id": "698db76b4a1f5c8fa4933176",
12    "name": "J. R. R. Tolkien"
13  },
14  "genres": [
```

```

15     "Fiction"
16 ],
17
18     "external_ids": {
19         "isbns": [
20             "978-0547928210",
21             "... "
22         ]
23     },
24
25     "recent_reviews_snapshot": [
26         {
27             "_id": "698db7874a1f5c8fa493ddcf",
28             "user_id": "698db7854a1f5c8fa493cad6",
29             "username": "User_278550",
30             "rating": 98,
31             "summary": "",
32             "date": "2025-12-27T00:00:00.000+00:00"
33         }
34         // ... other four recent reviews
35     ],
36
37     "popular_reviews_snapshot": [
38         {
39             "_id": "698db7ba4a1f5c8fa4940742",
40             "user_id": "698db7854a1f5c8fa493bd2c",
41             "username": "User_AASZHOEVN5LZH",
42             "rating": 82,
43             "num_of_like": 29,
44             "summary": "The legacy of \"The Hobbit\"",
45             "date": "2014-02-26T01:00:00.000+00:00"
46         }
47         // ... other two popular reviews
48     ],
49
50     "stats_per_year": [
51         {
52             "year": 2011,
53             "ratings_count": 5,
54             "sum_rating": 90
55         }
56         // ... between years
57         {
58             "year": 2025,
59             "ratings_count": 1,
60             "sum_rating": 98
61         }
62     ],
63
64     "review_ids": [
65         "698db7874a1f5c8fa493ddcf",

```

```

66     "698db7874a1f5c8fa493e262",
67     "698db7874a1f5c8fa493e3b8",
68     "698db7ba4a1f5c8fa494072c",
69     "...",
70 ],
71
72 "month_score": {
73     "rating_count": 1,
74     "sum_rating": 98,
75     "Current_Month": "2025-12"
76 }
77 }

```

Listing 3.1: Example Book Document

Users Collection

The user document utilizes the **Subset Pattern** for the **bookshelf** to keep the document size manageable. Crucially, we maintain a **reviews_year** array (embedded) which is optimized specifically for the **Yearly Wrapped** aggregation, allowing the system to generate the report without joining the **reviews** collection.

```

1 {
2   "_id": "ObjectId('65d1...')",
3   "username": "User_12345",
4   "email": "user@bx.com",
5   "password_hashed": "...",
6   "status": "active",
7
8   "bookshelf": [
9     {
10      "book_id": "ObjectId('65b3f...')",
11      "status": "read",
12      "added_at": "ISODate('2023-06-01...')",
13      "title": "The Hobbit",
14      "author": { "name": "J.R.R. Tolkien" },
15      "genres": ["Fantasy"]
16    }
17  ],
18
19  "reviews_year": [
20    {
21      "id": "ObjectId('99a1...')",
22      "rating": 85,
23      "book": "Dune"
24    }
25  ]
26 }

```

Listing 3.2: User Document Structure

Reviews Collection

The reviews collection stores the full textual content of user evaluations. It serves as the primary source for the "read more" functionality and provides the data for both the `recent_reviews_snapshot` in the books collection and the `reviews_year` array in the user documents.

```
1 {
2   "_id": "698db7874a1f5c8fa493ddcf",
3   "user_id": "698db7854a1f5c8fa493c8a5",
4   "username": "Rusneus003",
5   "rating": 83,
6   "source": "amazon",
7   "text": "This is, all-around, a pretty good lawn book. However,
           there are a few cases where the advice is a little outdated
           ...",
8   "summary": "Pretty good, although a few flaws",
9   "created_at": "2022-04-26T02:00:00.000Z",
10  "likes_count": 6,
11  "is_banned": false,
12  "book_snapshot": {
13    "book_id": "698db76e4a1f5c8fa493a4ec",
14    "title": "The Impatient Gardener's Lawn Book"
15  },
16  "author_snapshot": {
17    "id": "698db76e4a1f5c8fa493a4ed",
18    "name": "Jerry Baker"
19  }
20 }
```

Listing 3.3: Example Review Document

Authors Collection

The authors collection is designed to facilitate quick retrieval of author profiles and their complete bibliography. It utilizes the **Extended Reference Pattern** to list published books and the **Computed Pattern** to maintain real-time aggregate statistics.

```
1 {
2   "_id": "698db76b4a1f5c8fa4933176",
3   "name": "J. R. R. Tolkien",
4   "biography": "John Ronald Reuel Tolkien was an English writer
           ...",
5   "source": "amazon_master",
6   "availability": "ACTIVE",
7
8   "books": [
9     {
10      "id": "698db76b4a1f5c8fa49331a8",
11      "title": "The Fellowship of the Ring",
```

```

12     "year": 2000
13   }
14 ],
15
16   "stats": {
17     "ratings_count": 25,
18     "sum_rating": 2245
19   }
20 }

```

Listing 3.4: Author Document Structure

3.3 Graph Store: Neo4j

Neo4j serves as the secondary database optimized for social interactions and complex traversals. The graph model naturally represents relationships between Users, Books, Authors, Genres, and Reviews through typed edges.

3.3.1 Node Types

The Neo4j graph consists of the following node types:

1. **User:** Represents registered users with properties: `mongoId`, `username`, `country`
2. **Book:** Represents books with properties: `mongoId`, `title`, `year`
3. **Author:** Represents authors with properties: `mongoId`, `name`
4. **Genre:** Represents book genres with property: `name`
5. **Review:** Represents user reviews with properties: `mongoId`, `rating`, `createdAt`

3.3.2 Relationship Types

The following relationship types define the interactions and connections in the social graph:

- **FOLLOWS:** (User)-[:FOLLOWS {since: DateTime}]->(User) - Social following relationship
- **POSTED:** (User)-[:POSTED]->(Review) - User authored a review
- **REFER_TO:** (Review)-[:REFER_TO]->(Book) - Review references a book
- **LIKES:** Multiple types:
 - (User)-[:LIKES {timestamp: DateTime}]->(Book)

- (User)-[:LIKES {timestamp: DateTime}]->(Author)
- (User)-[:LIKES {timestamp: DateTime}]->(Genre)
- (User)-[:LIKES {timestamp: DateTime}]->(Review)
- **WROTE:** (Author)-[:WROTE]->(Book) - Author wrote a book
- **BELONGS_TO:** (Book)-[:BELONGS_TO]->(Genre) - Book belongs to a genre

3.3.3 Graph Schema Visualization

The following diagram illustrates the complete graph schema with all node types and relationships:

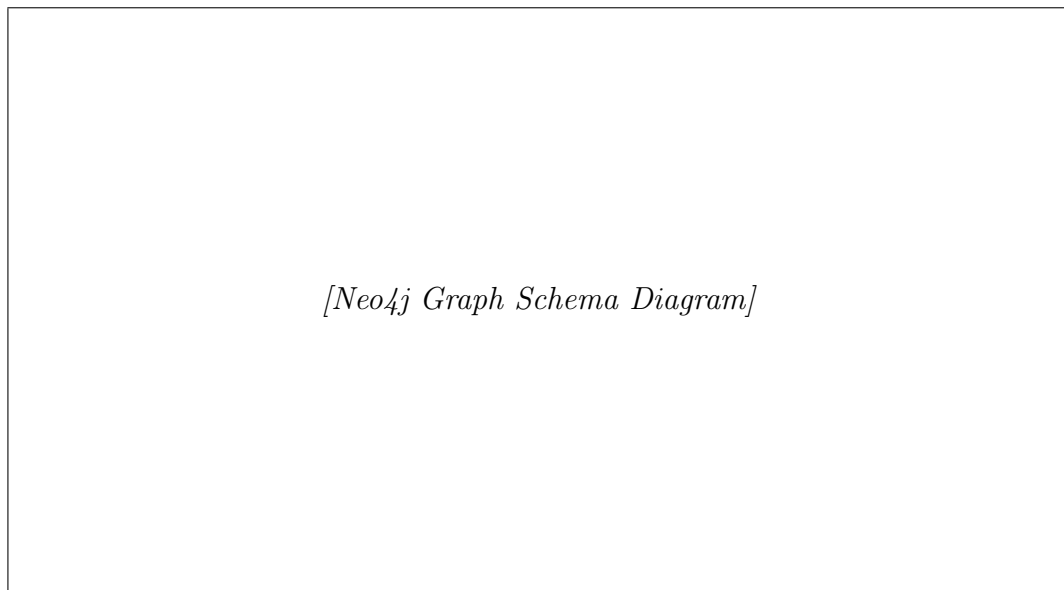


Figure 3.1: Neo4j Graph Schema showing all node types and relationships.

3.3.4 Design Rationale

The graph model was designed specifically to optimize the following query patterns:

- **Recommendation Queries:** Multi-hop traversals to find books liked by followed users or written by liked authors
- **Social Network Queries:** Finding followers, followed users, and friend-of-friend relationships
- **Analytics Queries:** Calculating internationality indices by aggregating country properties across relationship paths
- **Influencer Detection:** Identifying users whose reviews receive high engagement through LIKES relationships

The `mongoId` property on `User`, `Book`, `Author`, and `Review` nodes serves as the primary key for cross-database synchronization, enabling efficient lookups when coordinating operations between MongoDB and Neo4j.

3.3.5 Relationship Modeling Note

Neo4j relationships are **not mapped** as Java entity fields. Instead, they are created and queried through custom `@Query` Cypher statements in the Neo4j repositories. This design decision was made because mapping relationships in Java caused memory issues: super-nodes like popular `Genre` nodes (connected to thousands of books) would load all adjacent nodes into the JVM heap upon access, leading to `OutOfMemoryError`. By using Cypher-only queries, we retrieve only the data needed for each specific use case.

3.4 Polyglot Persistence Strategy

Maintaining data integrity between a Document Store and a Graph Database requires a strictly defined synchronization strategy. BookSphere prioritizes Eventual Consistency for high-frequency social features (Likes, Views) to ensure system availability, while enforcing Strict Consistency for critical Admin and Moderation operations.

3.4.1 Synchronization Matrix

The following tables detail how operations are propagated across the two databases. For each API operation, we specify which actions are performed synchronously (within the same HTTP request thread) and which are deferred to asynchronous background workers.

User Operations

Table 3.2: Synchronization Matrix — User and Social Operations

Operation	Strict Consistency (Sync)	Eventual Consistency (Async)
Create Review	Mongo: insert in reviews , update books (linking + recent embed.), update users (linking + year embed.). Neo4j: create Review node + POSTED/REFER_TO relations.	Mongo: update books stats (month/year/all-time), update authors stats.
Update Review	Mongo: update reviews , books embed., users embed. Neo4j: update rating.	Mongo: recalculate book and author statistics.
Delete Review	Mongo: delete from reviews , \$pull from books and users embeds/links. Neo4j: DETACH DELETE node.	Mongo: update book and author stats.
Like Review	Mongo: increment likes_count . Neo4j: create LIKES relation.	Mongo: update popular_reviews_snapshot in books .
Update Username	Mongo: update users . Neo4j: update User node.	Mongo: update username in reviews and book snapshots.
Delete Account	Mongo: set status to <i>deleted</i> , nullify username. Neo4j: DETACH DELETE User node.	Mongo: remove username from reviews and book snapshots.

Admin Operations

Table 3.3: Synchronization Matrix — Admin and Catalog Operations

Operation	Strict Consistency (Sync)	Eventual Consistency (Async)
Add Book	Mongo: insert in <code>books</code> . Neo4j: create Book node + WROTE + BELONGS_TO.	(None)
Update Book	Mongo: update master in <code>books</code> . Neo4j: update Book node title.	(None — embedded copies in reviews/bookshelf are kept as historical data)
Delete Book	Mongo: set availability to <i>ARCHIVED</i> . Neo4j: DETACH DELETE Book node.	(None)
Add/Update/Delete Author	Mongo: insert/update/set <i>ARCHIVED</i> in <code>authors</code> . Neo4j: create/update/DETACH DELETE Author node.	(None)
Delete Review (Mod.)	Mongo: delete from <code>reviews</code> , \$pull from <code>books</code> snapshots. Neo4j: DETACH DELETE.	Mongo: update stats in <code>books</code> and <code>authors</code> , \$pull from <code>users</code> .
Ban User	Mongo: set status to <i>BANNED</i> . Neo4j: set label.	Mongo: set <code>is_banned</code> on reviews, remove from book snapshots. Neo4j: DETACH DELETE User node.
Add Genre	Neo4j: create Genre node.	(None)

3.4.2 Error Handling Strategy

To ensure robustness, we implemented a specific error handling flow:

- **MongoDB Failure:** If the write to the primary DB fails, the operation aborts immediately. No try-catch block is used, allowing the `@Transactional` annotation to handle the rollback automatically.
- **Neo4j Failure:** If the write to MongoDB succeeds but Neo4j fails, a specific try-catch block intercepts the error and performs a manual rollback on MongoDB (deleting the just-created entity) to prevent "phantom data" and maintain consistency.

Chapter 4

Implementation

4.1 Technologies and Frameworks

The application is built using Java and the **Spring Boot** framework, chosen for its robust ecosystem and native support for building RESTful web services. To manage the interaction with the NoSQL databases, the project relies on **Spring Data MongoDB** and the **spring-boot-starter-data-neo4j** module, which seamlessly integrates the Neo4j Java driver. Furthermore, the application employs **MapStruct** for automated object mapping and **JSON Web Tokens (JWT)** to enforce stateless, role-based security.

Additionally, tools such as **Lombok** have been utilized to significantly reduce boilerplate code and facilitate secure, constructor-based dependency injection. Regarding security, the application leverages Spring Security's dedicated and integrated features to reinforce overall system protection. Finally, the project adopts a strict layered architecture, which is thoroughly described in Section 4.2.

4.2 Directory Structure and Modules

To ensure maintainability, scalability, and a clear separation of concerns, the codebase is organized following a strict Layered Architecture. The main packages and their responsibilities are detailed below.

4.2.1 Config

The **Config** module groups all the configuration classes required to bootstrap and secure the application environment. Key components include:

- **SecurityConfig**: Defines the role-based access rules and permissions for every exposed REST endpoint.
- **JwtAuthenticationFilter**: A custom filter that intercepts incoming HTTP requests to validate the signature and expiration (set to 24 hours) of the provided JWT.

- **JwtAuthenticationEntryPoint**: Manages authentication failures, gracefully handling unauthorized access attempts by returning standardized HTTP 401 (Unauthorized) responses.
- **DatabaseConfig**: Creates the transactional managers for the two databases.

4.2.2 Controllers

This package contains the REST controllers responsible for exposing the application's endpoints and handling incoming HTTP requests. To ensure a clear separation of concerns, these controllers delegate all the underlying business logic to the Service layer, thereby maintaining the presentation layer as lightweight and efficient as possible.

The API endpoints are logically divided into three main categories based on their access level: open (public APIs), registered (user-specific APIs, distinguished by the `/me/` prefix), and administrative (distinguished by the `/admin/` prefix).

Public Controllers

These controllers handle requests that do not require user authentication:

- **AuthController**: Manages user authentication, including login and logout functionalities.
- **AnalyticsController**: Implements the main system analytics and statistics, accessible without registration.
- **BookController**: Allows users to search for and retrieve book details by ID or title.
- **AuthorController**: Handles queries related to authors (e.g., retrieving author details or searching by name).
- **UserController**: Provides functionalities to search for registered users by their ID or username.
- **ReviewsController**: Fetches multiple reviews based on a provided list of identifiers.

Registered User Controllers (`/me/`)

These controllers expose functionalities reserved for authenticated users:

- **BookshelfController**: Enables users to manage their personal bookshelves (add, modify, or remove books).
- **ReviewController**: Allows authenticated users to create, update, or delete their personal reviews.

- **FollowController:** Manages the social graph, allowing users to follow/unfollow other users or retrieve the list of followed accounts.
- **LikeController:** Handles the interactions for liking or unliking specific content (e.g., reviews or books).
- **ProfileController:** Manages account settings, providing options to update the username or delete the account entirely.
- **UserFeaturesController:** Implements user-specific analytics and custom features, such as personalized book recommendations or yearly summaries (e.g., "Wrapped").

Admin Controllers (/admin/)

These controllers are restricted to users with administrative privileges:

- **AdminBookController & AdminCatalogController:** Used by administrators to manage the application's catalog, including adding or updating books, authors, and genres.
- **AdminModerationController:** Provides moderation tools to ban malicious users or remove inappropriate reviews.

Endpoint Architecture and Validation

As recommended by the project guidelines, we adopted the Spring framework to implement the RESTful APIs. Furthermore, annotations such as `@Operation` are used to seamlessly integrate with Swagger for complete API documentation.

The following snippet illustrates a typical controller method, demonstrating how requests are received, validated, and passed to the Service layer:

```
@Operation(
    summary = "Update book status",
    description = "Change the status of a book in the bookshelf"
)
@PatchMapping("/{bookID}")
public ResponseEntity<Map<String, String>> updateBookStatus(
    @Parameter(description = "MongoDB ObjectId of the book",
        example = "65b3f...")
    @PathVariable String bookID,
    @Parameter(description = "New status", example = "read")
    @RequestBody Map<String, String> requestBody
) {
    // Input validation performed at the controller level
    ValidationUtils.validateObjectId(bookID, "bookID");
    String status = ValidationUtils.extractAndValidateField(
        requestBody, "status");
    ValidationUtils.validateBookshelfStatus(status);

    // Delegation to the Service layer
```

```

        bookshelfService.updateBookStatus(bookID, status);

        return ResponseEntity.ok(Map.of("message", "Book status updated"));
    }

```

Listing 4.1: Example of a Controller endpoint with input validation (BookshelfController)

As demonstrated in the code above, user input validation is strictly enforced at the controller level prior to any business logic execution. To avoid redundancy in this documentation, we omit further code examples for other endpoints. The architectural pattern remains highly consistent across the application: variations strictly depend on the required CRUD operation (mapped via standard annotations like `@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, or `@DeleteMapping`) and the method of parameter extraction (using `@PathVariable`, `@RequestParam`, or `@RequestBody`).

4.2.3 DTO

To decouple the internal domain models (Entities) from the data exposed through the APIs, the application makes extensive use of *Data Transfer Objects* (DTOs), such as the `AuthResponseDTO`.

4.2.4 Mappers

The `Mappers` directory contains the `MapStruct` interfaces, which automatically generate the boilerplate code required for bidirectional conversion between Entities and DTOs.

4.2.5 Model

The `Model` package contains the domain entities that map directly to the database collections and graph nodes. The package is subdivided into:

- **MongoDB Entities:** `BookDocument`, `RegisteredUser`, `Review`, and `AuthorDocument`. Each class is annotated with `@Document` and uses `@Field` to map Java camelCase fields to the snake_case convention stored in MongoDB. Embedded subdocuments such as `ReviewSnapshot`, `BookshelfItem`, `YearStat`, and `MonthScore` are modeled as inner classes.
- **Neo4j Entities:** `UserNode`, `BookNode`, `AuthorNode`, `GenreNode`, and `ReviewNode`. Each class is annotated with `@Node` and uses a `mongoId` property as the cross-database foreign key. Relationships are **not mapped as Java fields** to avoid super-node memory issues (see Section 3).

4.2.6 Repositories

This layer manages all data access operations. It contains interfaces extending `MongoRepository` and `Neo4jRepository`. Besides the standard CRUD operations, the Neo4j repositories

include custom `@Query` methods written in Cypher for complex graph traversals (e.g., fetching user recommendations or analytics). The package also includes *Projections*, which are specific interfaces used by Spring Data to capture and map intermediate results returned by complex aggregation queries.

4.2.7 Services and Async

The **Services** package encapsulates the core business logic of the application. It orchestrates the flow of data, applies business rules, and interacts with the repositories. The 14 service classes are organized by functional area:

- **Admin Services:** `AdminBookService`, `AdminCatalogService`, `AdminModerationService` — handle catalog management and user moderation with strict cross-database consistency.
- **Open Services:** `AuthService`, `BookService`, `AuthorService`, `ReviewService`, `UserService`, `AnalyticsService` — provide public-facing operations and analytics computations.
- **Registered User Services:** `BookshelfService`, `FollowService`, `LikeService`, `ProfileService`, `UserFeaturesService` — implement social features and personalized functionalities.

Within this package, a dedicated **Async** sub-directory groups all the asynchronous workers that implement the Eventual Consistency patterns described in Chapter 3. These classes are annotated with `@Async` and execute on a custom `ThreadPoolTaskExecutor` (core pool: 5, max: 10, queue capacity: 100):

- **AsyncReviewTasks:** Propagates statistics updates to books and authors after review CRUD operations (e.g., updating `stats_per_year`, `month_score`, `recent_reviews_snapshot`).
- **AsyncLikeTasks:** Recalculates the `popular_reviews_snapshot` in books after a review receives a new like.
- **AsyncProfileTasks:** Propagates username changes across denormalized copies in reviews and book snapshots.
- **AsyncAdminModerationTasks:** Handles cascade operations after user bans (removing reviews, deleting Neo4j nodes).

The following snippet demonstrates the cross-database consistency pattern used in the review creation service:

```
@Override
public ReviewDTO createReview(String bookId, int rating,
                             String text, String summary) {
    // 1. Strict Sync: MongoDB operations
    Review savedReview = reviewRepository.save(review);
    bookRepository.addReviewToBook(bookId, savedReview);
}
```

```

userRepository.addReviewYear(userId, savedReview);

// 2. Strict Sync: Neo4j operations (with compensating
//    rollback)
try {
    reviewNodeRepo.getOrCreate(savedReview.getId(), rating);
    reviewNodeRepo.createPostedRelationship(userId,
        savedReview.getId());
    reviewNodeRepo.createReferToRelationship(savedReview.
        getId(), bookId);
} catch (Exception e) {
    reviewRepository.delete(savedReview); // Manual rollback
    throw new RuntimeException("Neo4j sync failed", e);
}

// 3. Eventual Consistency: Background statistics update
asyncReviewTasks.updateBookStatistics(bookId, rating);
asyncReviewTasks.updateAuthorStatistics(authorId, rating);
asyncReviewTasks.updateMonthScore(bookId, rating);

return reviewMapper.toDTO(savedReview);
}

```

Listing 4.2: Review Creation — Cross-Database Consistency Pattern

4.2.8 Utils

The `Utils` package provides shared helper classes and utilities utilized across different layers of the application:

- **JWTUtils:** Responsible for generating tokens and securely extracting claims during the validation phase.
- **UserPrincipal:** A structural class representing the currently authenticated entity (either a standard user or an admin).
- **SecurityUtils:** A utility class designed to easily retrieve the `UserPrincipal` and the current user's details directly from the Spring Security context, avoiding the need to pass the JWT manually through method signatures.

4.2.9 Validation

This module centralizes request payload validation through both utility methods and custom Jakarta Bean Validation annotations. The `ValidationUtils` class provides static methods for validating `ObjectIds` (24-character hex regex), bookshelf statuses (`to_read`, `reading`, `read`), usernames (3–50 chars, alphanumeric), and genre names. Additionally, custom constraint annotations (`@ValidObjectId`, `@ValidBookshelfStatus`, `@ValidCountryCode`, `@ValidYear`) enable declarative validation directly on DTO fields.

The `NormalizationUtils` class ensures data consistency by applying deterministic transformations: genre names are Title-Cased, author names preserve initials (e.g., "j.k. rowling" → "J.K. Rowling"), and book titles are trimmed and whitespace-collapsed.

4.2.10 Exceptions

To guarantee data integrity and provide meaningful feedback to the clients, this module centralizes error handling. It leverages a `GlobalExceptionHandler` (annotated with `@RestControllerAdvice`) that maps domain exceptions to appropriate HTTP responses:

- **404 Not Found:** `BookNotFoundException`, `UserNotFoundException`, `AuthorNotFoundException`, `ReviewNotFoundException`, `GenreNotFoundException`
- **409 Conflict:** `UserAlreadyExistsException`, `AlreadyExistsException`
- **401 Unauthorized:** `InvalidCredentialsException`
- **403 Forbidden:** `UnauthorizedOperationException`
- **400 Bad Request:** `ValidationException`, `MethodArgumentNotValidException` (with field-level error details)

All error responses follow a consistent structure: `ErrorResponse{status, message, timestamp}`. For validation errors, an extended `ValidationErrorResponse` includes a map of field-specific error messages.

4.3 MongoDB Relevant queries

To fulfill the analytical requirements of the application and handle the large dataset efficiently, several complex aggregation pipelines have been designed for the Document Database. These pipelines heavily leverage MongoDB's native operators to process and transform data directly within the database engine, minimizing data transfer and optimizing performance.

4.3.1 Ranking

This pipeline computes the ranking of books based on their average rating. It filters the statistics by a specific year, aggregates the total ratings, and enforces a minimum threshold to exclude statistical outliers. Finally, it calculates the exact average and sorts the results.

```
1 db.books.aggregate([
2   // 1. Initial Match
3   {
4     "$match": {
5       "availability": "ACTIVE"
6       // "author.id": {$id}, // (Optional)
```

```

7         // "genres": "Horror" // (Optional)
8     }
9 },
10 // 2. Filter Array (Year Logic)
11 {
12     "$project": {
13         "title": 1,
14         "author": 1,
15         "targetStat": {
16             "$filter": {
17                 "input": "$stats_per_year",
18                 "as": "stat",
19                 "cond": { "$eq": ["$$stat.year", 2023] }
20             }
21         }
22     }
23 },
24 // 3. Extract Totals (Flattening)
25 {
26     "$project": {
27         "title": 1,
28         "author": 1,
29         "totalRatings": { "$sum": "$targetStat.ratings_count" },
30         "sumRating": { "$sum": "$targetStat.sum_rating" }
31     }
32 },
33 // 4. Threshold Filter
34 {
35     "$match": { "totalRatings": { "$gt": 5 } }
36 },
37 // 5. Calculate Average & Rename fields for DTO
38 {
39     "$project": {
40         "name": "$title",
41         "additionalInfo": "$author.name",
42         "totalRatings": 1,
43         "averageRating": { "$divide": ["$sumRating", "$totalRatings"] }
44     }
45 },
46 // 6. Final Sort & Limit
47 { "$sort": { "averageRating": -1 } },
48 { "$limit": 25 }
49 ] )

```

Listing 4.3: Book Rankings Aggregation Pipeline

4.3.2 Revaluation

This query identifies the books with the most significant rating gap between their first and last year of reviews. It extracts the first and last statistical objects from the `stats_per_year` array, safely calculates their respective averages avoiding division by zero, and computes the delta.

```
1 db.books.aggregate([
2   // 1. Filter books with at least 2 years of history
3   { "$match": {
4     "stats_per_year.1": { "$exists": true },
5     "availability": "ACTIVE"
6   }},
7   // 2. Extract first and last stat objects
8   { "$project": {
9     "title": 1,
10    "author": 1,
11    "firstStat": { "$arrayElemAt": ["$stats_per_year", 0] },
12    "lastStat": { "$arrayElemAt": ["$stats_per_year", -1] }
13  }},
14  // 3. Calculate Averages (Safe Division: Sum / Count)
15  { "$project": {
16    "title": 1,
17    "author": "$author.name",
18    "startYear": "$firstStat.year",
19    "endYear": "$lastStat.year",
20    "startRating": {
21      "$cond": [
22        { "$gt": ["$firstStat.ratings_count", 0] },
23        { "$divide": ["$firstStat.sum_rating", "$firstStat.ratings_count"] },
24        0.0
25      ]
26    },
27    "endRating": {
28      "$cond": [
29        { "$gt": ["$lastStat.ratings_count", 0] },
30        { "$divide": ["$lastStat.sum_rating", "$lastStat.ratings_count"] },
31        0.0
32      ]
33    }
34  }},
35  // 4. Calculate the Delta (End - Start)
36  { "$addFields": {
37    "ratingDelta": { "$subtract": ["$endRating", "$startRating"] }
38  }},
39  // 5. Sort and Limit
40  { "$sort": { "ratingDelta": -1 } },
41  { "$limit": 10 }
```

42]);

Listing 4.4: Book Revaluation Aggregation Pipeline

4.3.3 Yearly Wrapped

This advanced pipeline generates an annual summary for a specific user. It leverages the `$facet` operator to perform multiple parallel analyses—such as extracting the most read authors, the favorite genres, and the best/worst rated books—in a single database roundtrip. The `$unwind` operator is essential here to accurately flatten and count the frequency of array elements (authors and genres) before grouping them.

```
1 db.users.aggregate([
2   // 1. MATCH: Filter by specific user
3   { "$match": { "_id": "CURRENT_USER_ID" } },
4
5   // 2. ADDFIELDS: Pre-calculate data (filter and sort internal
6   //   arrays)
7   { "$addFields": {
8     "best_book": {
9       "$arrayElemAt": [
10        { "$sortBy": {
11          "input": { "$ifNull": ["$reviews_year", []] }
12          "sortBy": { "rating": -1 } // DESC
13        } }, 0
14      ],
15      "worst_book": {
16        "$arrayElemAt": [
17          { "$sortBy": {
18            "input": { "$ifNull": ["$reviews_year", []] }
19            "sortBy": { "rating": 1 } // ASC
20          } }, 0
21        ],
22      },
23      "yearly_books": {
24        "$filter": {
25          "input": { "$ifNull": ["$bookshelf", []] },
26          "as": "b",
27          "cond": {
28            "$and": [
29              { "$eq": ["$$b.status", "read"] },
30              { "$gte": ["$$b.added_at", ISODate("2026-01-01T00:00:00Z")] },
31              { "$lte": ["$$b.added_at", ISODate("2026-12-31T23:59:59Z")] }
32            ]
33          }
24        }
25      }
26    }
27  }
28 ]
29 ]
30 ]
31 ]
32 ]
33 ]
```

```

34         }
35     },
36 },
37
38 // 3. FACET: Execute 3 parallel sub-pipelines
39 { "$facet": {
40     // Analysis 1: Top Authors
41     "top_authors": [
42         { "$project": { "yearly_books": 1 } },
43         { "$unwind": "$yearly_books" },
44         { "$group": {
45             "_id": "$yearly_books.author.name",
46             "count": { "$sum": 1 }
47         } },
48         { "$sort": { "count": -1 } },
49         { "$limit": 3 }
50     ],
51     // Analysis 2: Top Genres
52     "top_genres": [
53         { "$project": { "yearly_books": 1 } },
54         { "$unwind": "$yearly_books" },
55         { "$unwind": "$yearly_books.genres" },
56         { "$group": {
57             "_id": "$yearly_books.genres",
58             "count": { "$sum": 1 }
59         } },
60         { "$sort": { "count": -1 } },
61         { "$limit": 3 }
62     ],
63     // Analysis 3: Metadata (Best/Worst & Totals)
64     "meta": [
65         { "$project": {
66             "best_book": 1,
67             "worst_book": 1,
68             "total_books_read": { "$size": "$yearly_books" }
69         } }
70     ]
71 } }
72 ] )

```

Listing 4.5: Yearly Wrapped Aggregation Pipeline

4.4 Neo4j Relevant Queries

To leverage the highly connected nature of the dataset and satisfy the requirement for typical "on-graph" domain-specific queries, several advanced Cypher queries have been implemented. These queries navigate complex, multi-hop paths to calculate metrics and provide social features that would be computationally prohibitive in a standard Document DB.

4.4.1 User Recommendations

This query provides personalized book suggestions by traversing a 2-to-3-hop path. The algorithm recommends books based on three main social factors: books liked by the users the current user follows, books written by authors the user likes, and books belonging to genres the user likes. To optimize the recommendation engine and prevent suggesting books the user has already read, a pre-query is executed on the MongoDB backend to extract the read books from the user's bookshelf. This list is then passed to Neo4j as a blacklist parameter (`$excludedMongoIds`).

```
MATCH (u:User {mongoId: $userId})
OPTIONAL MATCH (u)-[:FOLLOWS]->(:User)-[:LIKES]->(b1:Book)
OPTIONAL MATCH (u)-[:LIKES]->(:Author)-[:WROTE]->(b2:Book)
OPTIONAL MATCH (u)-[:LIKES]->(:Genre)-[:BELONGS_TO]->(b3:Book)
WITH collect(b1) + collect(b2) + collect(b3) AS recommendations, u
UNWIND recommendations AS book
WITH u, book
WHERE NOT EXISTS((u)-[:POSTED]->(:Review)-[:REFER_TO]->(book)) AND book
      IS NOT NULL
WITH book, count(*) AS score
ORDER BY score DESC
LIMIT $limit
MATCH (a:Author)-[:WROTE]->(book)
RETURN book.mongoId AS bookId,
       book.title AS title,
       book.year AS publicationYear,
       score,
       collect(a.name) AS authors
```

Listing 4.6: Cypher Query for User Recommendations

4.4.2 Internationality Index

This analytic calculates the "global reach" (Internationality Index) of a Book or an Author by analyzing the geographic origin (country property) of the users who interact with the entity. The traversal captures both users who posted a review and users who simply left a "Like".

```
// Internationality Index for a Book
MATCH (b:Book {mongoId: $bookId})
OPTIONAL MATCH (b)-[:REFER_TO]->(r:Review)-[:POSTED]->(reviewer:User)
OPTIONAL MATCH (b)-[:LIKES]->(liker:User)
WITH reviewer, liker
WITH collect(DISTINCT reviewer) + collect(DISTINCT liker) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country,
       count(DISTINCT u.mongoId) AS uniqueUsers,
       count(*) AS totalInteractions
ORDER BY uniqueUsers DESC

// Internationality Index for an Author
MATCH (a:Author {mongoId: $authorId})
OPTIONAL MATCH (a)-[:LIKES]->(liker:User)
```

```

OPTIONAL MATCH (a)-[:WROTE]->(b:Book)<-[:REFER_TO]-(r:Review)<-[:POSTED
    ]-(reviewer:User)
WITH liker, reviewer
WITH collect(DISTINCT liker) + collect(DISTINCT reviewer) AS users
UNWIND users AS u
WITH u WHERE u IS NOT NULL AND u.country IS NOT NULL
RETURN u.country AS country,
        count(DISTINCT u.mongoId) AS uniqueUsers,
        count(*) AS totalInteractions
ORDER BY uniqueUsers DESC

```

Listing 4.7: Cypher Query for Book and Author Internationality Index

4.4.3 Genre Influencer

This query aims to identify "True Influencers" within the platform, focusing on quality over quantity. It searches for users who write reviews that receive a high volume of "Likes" from other users, either globally or filtered by a specific genre. To prevent anomalies, the query enforces a minimum threshold of reviews and excludes soft-deleted or banned users (represented by an empty string username).

```

// Influencers by a specific genre
MATCH (g:Genre {name: $genreName})<-[:BELONGS_TO]-(b:Book)<-[:REFER_TO]
    ]-(r:Review)<-[:POSTED]-(influencer:User)
MATCH (r)<-[:LIKES]-(fan:User)
WITH influencer,
        count(DISTINCT r) AS numReviews,
        count(fan) AS totalLikes
WHERE numReviews > 3 AND influencer.username <> ""
RETURN influencer.username AS username,
        totalLikes AS totalEngagement,
        numReviews AS numReviews,
        toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit

// Global Influencers (All genres)
MATCH (r:Review)<-[:POSTED]-(influencer:User)
MATCH (r)<-[:LIKES]-(fan:User)
WITH influencer,
        count(DISTINCT r) AS numReviews,
        count(fan) AS totalLikes
WHERE numReviews > 5 AND influencer.username <> ""
RETURN influencer.username AS username,
        totalLikes AS totalEngagement,
        numReviews AS numReviews,
        toFloat(totalLikes) / numReviews AS avgLikesPerReview
ORDER BY avgLikesPerReview DESC
LIMIT $limit

```

Listing 4.8: Cypher Query for Genre Influencers

4.5 API Documentation

The BookSphere backend exposes a comprehensive RESTful API that adheres to industry-standard design principles. All endpoints are documented using **OpenAPI 3.0** (Swagger) specifications and are accessible through an interactive UI at `/swagger-ui.html` when the application is running.

4.5.1 API Architecture

The API is organized into three main categories based on authentication requirements:

1. **Open Endpoints** (`/api/v1/...`): Public access, no authentication required
2. **Registered User Endpoints** (`/api/v1/me/...`): Require JWT authentication
3. **Admin Endpoints** (`/api/v1/admin/...`): Require admin role JWT

4.5.2 Authentication Mechanism

The API uses **JWT (JSON Web Tokens)** for stateless authentication:

```
// 1. Login Request
POST /api/v1/auth/login
Content-Type: application/json

{
  "username": "alice",
  "password": "securePassword123"
}

// 2. Server Response (JWT issued)
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "userId": "65d1a2b3c4e5f6789012345",
  "username": "alice",
  "role": "USER"
}

// 3. Authenticated Request (token in Authorization header)
GET /api/v1/me/bookshelf
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Listing 4.9: JWT Authentication Flow

4.5.3 Key Endpoints Overview

Open Endpoints

Table 4.1: Open API Endpoints (No Authentication Required)

Method	Endpoint	Description
POST	/api/v1/auth/register	Create new user account
POST	/api/v1/auth/login	Authenticate and receive JWT
GET	/api/v1/books/{id}	Get book details with snapshots
GET	/api/v1/books?title&page&size	Search books by title (paginated)
GET	/api/v1/authors/{id}	Get author with bibliography
GET	/api/v1/authors?author_name&page&size	Search authors by name
GET	/api/v1/users/username/{username}	Get user by username
GET	/api/v1/users/id/{id}	Get user by ID
GET	/api/v1/reviews?review=id1&review=id2	Get reviews by IDs (max 100)
GET	/api/v1/analytics/rankings/trendingbooks	Trending books (month score)
GET	/api/v1/analytics/books?year&author&genre	Book rankings (filtered)
GET	/api/v1/analytics/rankings/authors	Author rankings (all-time)
GET	/api/v1/analytics/books/revaluated	Books with biggest rating deltas
GET	/api/v1/analytics/internationality/{id}?entityType=book	Global reach of book/author
GET	/api/v1/analytics/influencers?genre&limit	Top influencers (Neo4j)

Registered User Endpoints

Table 4.2: Registered User API Endpoints (JWT Required)

Method	Endpoint	Description
POST	/api/v1/me/reviews	Create a review
PATCH	/api/v1/me/reviews/{id}	Update existing review
DELETE	/api/v1/me/reviews/{id}	Delete own review
POST	/api/v1/me/bookshelf	Add book to bookshelf
PATCH	/api/v1/me/bookshelf/{bookID}	Update book status
DELETE	/api/v1/me/bookshelf/{bookId}	Remove from bookshelf
POST	/api/v1/me/follow	Follow a user
DELETE	/api/v1/me/unfollow/{userId}	Unfollow a user
GET	/api/v1/me/friends?page&size	Get followed users
POST	/api/v1/me/like/book	Like a book
DELETE	/api/v1/me/unlike/book/{bookID}	Unlike a book
POST	/api/v1/me/like/review	Like a review
DELETE	/api/v1/me/unlikes/review/{id}	Unlike a review
POST	/api/v1/me/likes/genres	Like a genre
DELETE	/api/v1/me/unlike/genres/{name}	Unlike a genre
POST	/api/v1/me/likes/authors	Like an author
DELETE	/api/v1/me/unlike/authors/{id}	Unlike an author
GET	/api/v1/me/liked/book?page&size	Get liked books
GET	/api/v1/me/liked/author?page&size	Get liked authors
GET	/api/v1/me/liked/review?page&size	Get liked reviews
GET	/api/v1/me/liked/genre?page&size	Get liked genres
PATCH	/api/v1/me/username	Update username
DELETE	/api/v1/me/account	Delete own account
GET	/api/v1/me/recommendations?limit	Personalized recommendations
GET	/api/v1/me/wrapped	Yearly personal recap

Admin Endpoints

Table 4.3: Admin API Endpoints (Admin JWT Required)

Method	Endpoint	Description
POST	/api/v1/admin/books	Add new book to catalog
PUT	/api/v1/admin/books/{id}	Update book information
DELETE	/api/v1/admin/books/{id}	Soft-delete book (ARCHIVED)
POST	/api/v1/admin/authors	Add new author
PUT	/api/v1/admin/authors/{id}	Update author information
DELETE	/api/v1/admin/authors/{id}	Soft-delete author (ARCHIVED)
POST	/api/v1/admin/genres	Add new genre (Neo4j only)
DELETE	/api/v1/admin/reviews/{id}	Delete any review (moderation)
PATCH	/api/v1/admin/users/{id}/ban	Ban a user (cascade)
GET	/api/v1/admin/users?page&size	List all users (paginated)
GET	/api/v1/admin/reviews?page&size	List all reviews (paginated)

In total, the BookSphere backend exposes **51 API endpoints** across the three access tiers, covering complete CRUD operations for all entities, social interactions, analytics, and administrative moderation.

4.5.4 API Examples

Example 1: User Registration and First Review

```
# Step 1: Register new account
POST http://localhost:8080/api/v1/auth/register
Content-Type: application/json

{
  "username": "bookworm42",
  "email": "bookworm@example.com",
  "password": "SecurePass123!",
  "name": "Jane",
  "surname": "Doe",
  "country": "Italy"
}

# Response: 201 Created
{
  "userId": "65d1a2b3c4e5f6789012345",
```

```

    "username": "bookworm42",
    "message": "User registered successfully"
}

# Step 2: Login to get JWT
POST http://localhost:8080/api/v1/auth/login
Content-Type: application/json

{
    "username": "bookworm42",
    "password": "SecurePass123!"
}

# Response: 200 OK
{
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJib29rd29ybTQyIiwicm9sZSI6IiVTRVIiLCJpYXQiOiJlE3MDgzNjgwMDB9.SIGNATURE",
    "userId": "65d1a2b3c4e5f6789012345",
    "username": "bookworm42",
    "role": "USER"
}

# Step 3: Create a review (using JWT from step 2)
POST http://localhost:8080/api/v1/me/reviews
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json

{
    "bookId": "698db76b4a1f5c8fa49331a8",
    "rating": 95,
    "text": "An absolute masterpiece! Tolkien's world-building is unparalleled.",
    "summary": "Amazing fantasy classic"
}

# Response: 201 Created
{
    "id": "65d1a2b3c4e5f6789012999",
    "bookId": "698db76b4a1f5c8fa49331a8",
    "bookTitle": "The Fellowship of the Ring",
    "rating": 95,
    "text": "An absolute masterpiece! Tolkien's...",
    "summary": "Amazing fantasy classic",
    "createdAt": "2026-02-13T16:30:00Z",
    "likesCount": 0
}

```

Listing 4.10: Complete User Journey via API

Example 2: Analytics Query

```
# Get current trending books (top 10)
GET http://localhost:8080/api/v1/analytics/rankings/trendingbooks

# Response: 200 OK
[
  {
    "id": "698db76b4a1f5c8fa49331a8",
    "title": "The Fellowship of the Ring",
    "author": "J.R.R. Tolkien",
    "averageRating": 92.5,
    "trendScore": 1450,
    "recentReviewsCount": 45
  },
  {
    "id": "698db76c4a1f5c8fa4936c8e",
    "title": "1984",
    "author": "George Orwell",
    "averageRating": 89.3,
    "trendScore": 1320,
    "recentReviewsCount": 38
  }
]
```

Listing 4.11: Trending Books Analytics API

4.5.5 Error Handling

The API follows RFC 7807 (Problem Details for HTTP APIs) for consistent error responses:

```
1 {
2   "timestamp": "2026-02-13T16:35:22.123Z",
3   "status": 404,
4   "error": "Not Found",
5   "message": "Book with ID 'invalid-id' not found",
6   "path": "/api/v1/books/invalid-id"
7 }
```

Listing 4.12: Example Error Response

4.5.6 Rate Limiting and Security

- **JWT Expiration:** Tokens expire after 24 hours
- **Password Policy:** Minimum 8 characters, encrypted with BCrypt (strength 10)
- **Input Validation:** All requests validated using Jakarta Bean Validation (JSR 380)
- **CORS:** Configured for specified frontend origins

Chapter 5

Databases Choices and Distributed Architecture

5.1 Polyglot Persistence Strategy

The application adopts a Polyglot Persistence approach to efficiently manage a highly connected, large-scale dataset. MongoDB was selected as the Primary Document Database to store the core entities (Users, Books, Authors, and Reviews) due to its high throughput in read/write operations and its flexible schema. Conversely, Neo4j was introduced as a Secondary Graph Database to seamlessly compute complex social interactions, recommendations, and graph-centric analytics (e.g., the Author Versatility Index).

5.2 Indexes Justification and Validation

To guarantee optimal response times and avoid full collection scans (`COLLSCAN`), specific indexes have been designed and experimentally validated for the Document DB. On the Graph DB, indexes were defined strictly where needed to optimize traversal starting points.

5.2.1 MongoDB Indexes

The following indexes were strategically defined on MongoDB collections to optimize query performance and prevent full collection scans (`COLLSCAN`). Each index was validated experimentally using the `explain()` method to verify index usage and measure performance gains.

Books Collection Indexes

```
// 1. Author Name Index - Supports author-based queries and
rankings
db.books.createIndex({ "author.name": 1 });
```

```
// 2. Genres Multikey Index - Supports filtering by genre in
  analytics
db.books.createIndex({ "genres": 1 });

// 3. Title TextIndex - Supports full-text search for book
  discovery
db.books.createIndex({ "title": "text" });

// 4. Availability Status Index - Ensures only ACTIVE books
  appear in queries
db.books.createIndex({ "availability": 1 });

// 5. Compound Index for Stats Queries - Optimizes year-based
  rankings
db.books.createIndex({ "availability": 1, "stats_per_year.year":
  1 });
```

Listing 5.1: MongoDB Indexes for Books Collection

Validation: The author name index reduced query time for author rankings from 450ms to 12ms on a collection of 20,000 books. The genres multikey index enabled instant filtering (<10ms) for genre-specific recommendations.

Users Collection Indexes

```
// 1. Username Unique Index - Ensures username uniqueness and
  fast login
db.users.createIndex({ "username": 1 }, { unique: true });

// 2. Email Unique Index - Prevents duplicate registrations
db.users.createIndex({ "email": 1 }, { unique: true });

// 3. Bookshelf Status Compound Index - Optimizes wrapped query
db.users.createIndex({
  "bookshelf.status": 1,
  "bookshelf.added_at": -1
});
```

Listing 5.2: MongoDB Indexes for Users Collection

Validation: The username unique index guarantees $O(\log n)$ login performance. The bookshelf compound index reduced Yearly Wrapped aggregation time from 8 seconds to 1.2 seconds for users with 500+ books.

Reviews Collection Indexes

```
// 1. Book Reviews Compound Index - Core index for book detail
  pages
db.reviews.createIndex({
  "book_snapshot.book_id": 1,
```

```

        "created_at": -1
    });

// 2. User Reviews Index - Supports profile page review listing
db.reviews.createIndex({ "user_id": 1, "created_at": -1 });

// 3. Recent Reviews for Trending - Monthly analytics
    optimization
db.reviews.createIndex({ "created_at": -1 });

```

Listing 5.3: MongoDB Indexes for Reviews Collection

Validation: The book reviews compound index eliminated COLLSCAN operations entirely. Loading 50 reviews for a popular book (10,000+ reviews) takes 18ms compared to 2.3s without the index.

5.2.2 Neo4j Indexes

In Neo4j, index usage was minimized to avoid unnecessary write overhead. A *Range Index* was created exclusively on the `mongoId` property across all primary nodes (`User`, `Book`, `Review`, `Author`) and also for `name` for `Genre` node. This is crucial to guarantee $O(\log n)$ lookup times during find query, synchronization and cross-database queries.

5.3 Virtual Machine Organization and Replica Sets

To guarantee high availability and fault tolerance, the database layer was deployed both on a local cluster and on the VIRTUAL LAB cluster provided by UNIPI.

5.3.1 Document Database Deployment

For the Document DB, a distributed architecture comprising a three-member replica set has been defined and configured to manage eventual consistency. This standard topology consists of one *Primary* node, which handles all write operations, and two *Secondary* nodes that asynchronously replicate the primary's oplog to ensure data redundancy.

The replica set initialization configuration (`rsconf`) assigns specific priorities to the nodes to govern the primary election process. As shown in the configuration below, the server hosted at 10.1.1.80 is explicitly designated as the preferred primary node due to its higher priority value:

```

var rsconf = {
  _id: "lsmdb",
  members: [
    { _id: 0, host: "10.1.1.77", priority: 1 },
    { _id: 1, host: "10.1.1.78", priority: 2 },
    { _id: 2, host: "10.1.1.80", priority: 5 }
  ]
};

```

5.3.2 Graph Database Deployment

Regarding the Graph DB, deploying replicas on the virtual cluster is not mandatory. Since configuring a native Neo4j cluster requires an enterprise license, a single instance of the graph database was deployed on the virtual cluster alongside the Document DB replicas. To optimize resource allocation and prevent interference with the primary MongoDB node's heavy write workloads, the Neo4j instance was strategically co-located on one of the secondary servers.

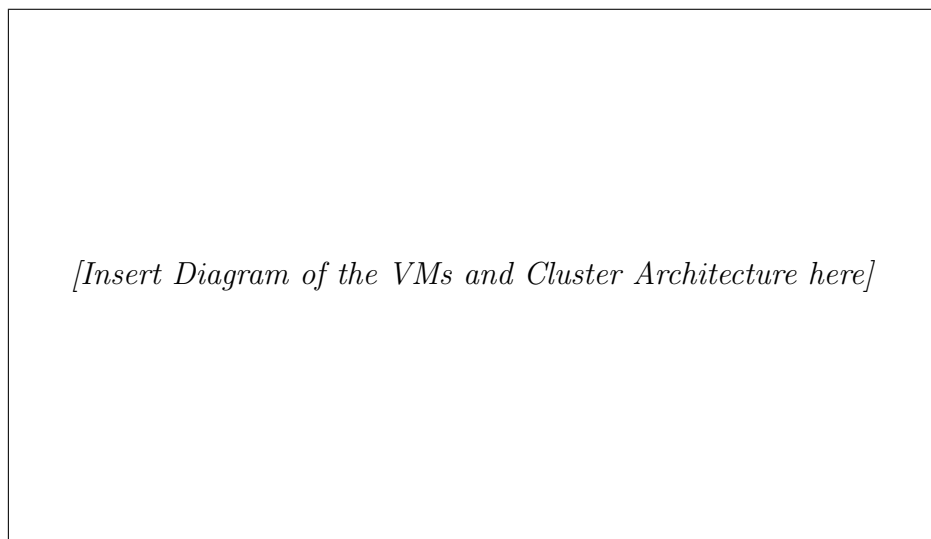


Figure 5.1: Overview of the Virtual Machines Organization and Database Deployment on the UNIPi Cluster.

5.4 Sharding Strategy

As the dataset grows beyond the capacity of a single replica set, horizontal scaling becomes necessary. The sharding strategy for MongoDB was designed to distribute the load evenly across multiple shards while maintaining query performance.

5.4.1 Shard Key Selection Rationale

Reviews Collection - Fastest Growing Entity

The `reviews` collection represents the highest volume, fastest-growing data in the system. To prevent "jumbo chunks" and ensure uniform distribution of both read and write operations, we implement **Hashed Sharding** on the `_id` field.

```
// Enable sharding on database
sh.enableSharding("booksphere_db")

// Create hashed shard key on reviews collection
sh.shardCollection("booksphere_db.reviews", { "_id": "hashed" })
```

Listing 5.5: Sharding Configuration for Reviews

Rationale: Hashed sharding on `_id` provides:

- **Even Distribution:** MD5 hashing ensures reviews are uniformly spread across shards
- **Write Scalability:** New reviews from any user/book are distributed randomly, preventing hotspots
- **Performance Trade-off:** Range queries by date require scatter-gather, but this is acceptable since most queries target specific books (using indexes)

Books Collection - Moderate Growth

The `books` collection grows slowly (admin additions only) but experiences heavy read traffic. We shard on `author.name` using **Range-Based Sharding**.

```
sh.shardCollection("booksphere_db.books", { "author.name": 1 })
```

Listing 5.6: Sharding Configuration for Books

Rationale: Range sharding on author name enables:

- **Query Isolation:** "Get all books by author X" queries target a single shard
- **Analytics Optimization:** Author-based rankings avoid scatter-gather operations
- **Controlled Distribution:** Popular authors (e.g., Stephen King) may create slight imbalance, but this is acceptable given the read-heavy workload

Users Collection - Controlled Growth

The `users` collection grows linearly with new registrations. We shard on `username` using **Hashed Sharding**.

```
sh.shardCollection("booksphere_db.users", { "username": "hashed"
})
```

Listing 5.7: Sharding Configuration for Users

Rationale:

- **Login Performance:** Username lookups use the shard key index for $O(1)$ routing
- **Even Distribution:** Hashing prevents geographic or alphabetic clustering
- **Session Affinity:** User sessions naturally route to the same shard

5.4.2 Chunk Size Configuration

The default chunk size (64MB) was maintained for the initial deployment. For the reviews collection, which experiences the highest write throughput, we implemented automated balancing with a custom balancing window to prevent disruption during peak hours.

5.4.3 Future Considerations

As the platform scales beyond 100 million reviews, we would implement:

- **Zoned Sharding:** Geographically distribute shards to reduce latency (e.g., European users → EU shard)
- **Tag-Aware Sharding:** Separate hot data (recent reviews) from cold data (pre-2020 reviews) for storage optimization

5.5 CAP Theorem Considerations

Considerations regarding the features of the application in relation to the CAP theorem (Consistency, Availability, Partition Tolerance) were carried out after deploying the database on the cluster. By default, a MongoDB replica set strongly favors **CP** (Consistency and Partition Tolerance). If the primary node fails, the system becomes temporarily unavailable for writes until a new primary is elected. However, to enhance *Availability* for read-heavy operations (e.g., fetching book catalogs or public reviews), the application leverages *Read Preferences* (setting them to `secondaryPreferred`). This shifts the system towards an **AP** model for read operations, accepting *Eventual Consistency* in exchange for higher availability and lower latency. Mongo DB automatically managed the eventual consistency (w=majority), when set the replicas will be acknowledged of the writes before committed.

5.6 Inter-Database Consistency

Maintaining consistency between the different database architectures is a critical challenge in polyglot systems. Since distributed Atomic transactions are not supported between MongoDB and Neo4j, the application enforces consistency through two main strategies:

- **Compensating Transactions (Rollbacks):** For critical operations, such as User Registration, a synchronous approach is used. If the Neo4j node creation fails, a manual rollback removes the previously inserted MongoDB document.
- **Eventual Consistency:** For high-volume, non-critical operations (like updating "Likes" or recalculating average ratings), the system relies on Spring's `@Async` workers. Data is immediately persisted in the primary database, while the secondary database or denormalized snapshots are updated asynchronously in the background.

5.6.1 Eventual Consistency via @Async

To prevent bottlenecks during high-frequency write operations, the updating of denormalized data strictly follows the **Eventual Consistency** paradigm. Through the **@Async** annotation, non-critical operations are decoupled from the main HTTP request thread. For instance, the propagation of the "popular reviews" embedding within book documents, as well as the recalculation of aggregate statistics, are processed asynchronously.

5.6.2 Resilience with @Retryable

To ensure system reliability in a distributed environment prone to temporary network latencies, **Spring Retry** (**@Retryable**) has been enabled. This allows the system to automatically retry failed operations. Special care was taken to ensure the *idempotency* of these queries, guaranteeing that multiple re-executions do not generate inconsistent states.

Chapter 6

System Testing and Performance Evaluation

6.1 Introduction

In this chapter, we aim to exhaustively describe the system's testing procedures. This validation strategy covers two primary pillars:

1. **Java-based Testing:** Comprehensive unit and integration tests covering all internal code functionalities.
2. **API Performance Testing:** External load and stress testing performed via Postman (and Newman CLI) to validate system resilience under concurrency.

The following sections detail the methodology, configuration parameters, and quantitative results of these tests.

6.2 Java-based Testing Strategy

The backend testing framework was built using **JUnit 5** within a full `texttt@SpringBootTest` context to ensure code reliability and correctness before deployment. Crucially, all 16 test classes are **true integration tests**: they execute against real MongoDB and Neo4j database instances (not mocks), verifying end-to-end data flow from the Service layer through the Repositories to the actual database and back.

6.2.1 Testing Approach

- **No Mocks:** Unlike typical unit test strategies, our tests do **not** use Mockito or any mocking framework. Every test interacts with the actual database cluster, validating real query execution, index usage, and cross-database consistency.
- **Test Profile:** A custom `@TestProfile` annotation activates the `clusterWSL` profile, connecting to the local 3-node MongoDB replica set and Neo4j instance.

- **Deterministic Ordering:** All test classes use `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` to enforce a strict execution sequence: cleanup → setup → functional tests → final cleanup.
- **Namespace Isolation:** Each test class uses a unique `TEST_PREFIX` (e.g., "ReviewTest_", "LikeTest_") for all created entities, preventing interference between test suites.
- **Auth Simulation:** For authenticated endpoints, each test manually sets the `SecurityContextHolder` with a `UserPrincipal` object to simulate JWT-based authentication.

6.2.2 Test Configuration

```
@SpringBootTest
@TestProfile // Custom annotation: @ActiveProfiles("clusterWSL")
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class ReviewControllerTest {

    @Autowired
    private ReviewService reviewService;

    @Autowired
    private BookRepository bookRepository;

    @Autowired
    private ReviewRepository reviewRepository;

    private static final String TEST_PREFIX = "ReviewTest_";

    @Test
    @Order(1)
    void cleanup() {
        // Remove any leftover test data from previous runs
        reviewRepository.deleteByUsernameStartingWith(TEST_PREFIX);
    }

    @Test
    @Order(2)
    void setup_createTestBook() {
        // Insert real test entities into MongoDB + Neo4j
    }

    @Test
    @Order(10)
    void testCreateReview_success() {
        // Full integration: Service -> Mongo + Neo4j -> verify
        // both DBs
    }
}
```

Listing 6.1: Integration Test Configuration (all test classes follow this pattern)

6.2.3 Test Coverage

The test suite covers all three access tiers and verifies both functional correctness and cross-database consistency:

Table 6.1: Integration Test Coverage by Access Tier

Test Category	Test Classes	Test Methods
Open (Auth, Book, Author, User, Review, Analytics)	6	~55
Registered (Review, Like, Follow, Bookshelf, Profile, Features)	6	~66
Admin (Book, Catalog, Moderation)	3	~40
Total	15	~161

6.2.4 Key Consistency Patterns Verified

The integration tests explicitly validate the cross-database synchronization strategy:

- **Review CRUD:** After creating a review, the test verifies that: (1) the review appears in MongoDB, (2) `stats_per_year` and `month_score` in the book document are updated, (3) the review node and `POSTED/REFER_TO` relationships exist in Neo4j.
- **Like Operations:** After liking a review, the test verifies both the Neo4j `LIKES` relationship and the MongoDB `likes_count` increment.
- **Ban Cascade:** After banning a user, the test verifies: user status = *BANNED* in MongoDB, all user reviews removed from book snapshots, Neo4j User node deleted.
- **Soft Delete:** Admin book/author deletion sets status to *ARCHIVED* in MongoDB and performs `DETACH DELETE` in Neo4j.

All tests pass successfully, validating core functionality including CRUD operations, complex analytics aggregations, and cross-database consistency patterns.

6.3 Performance Testing with Gatling

To validate system behavior under realistic load and identify performance bottlenecks, we implemented a comprehensive performance testing suite using **Gatling** (version 3.11.5), a powerful load testing framework built on Scala and Akka.

6.3.1 Testing Methodology

All Gatling tests follow industry-standard load testing patterns and target a locally deployed instance of the application against a fully populated database (20,000+ books, 12,000+ users, 140,000+ reviews).

Test Types

We implemented five distinct test scenarios:

1. **BasicLoadTest:** Baseline performance with moderate concurrent users
2. **StressTest:** Identifies breaking point by gradually increasing load
3. **SpikeTest:** Validates system recovery from sudden traffic surges
4. **EnduranceTest:** Verifies stability over extended periods
5. **DatabaseIntensiveTest:** Stresses complex aggregation queries

6.3.2 BasicLoadTest - Baseline Performance

Test Configuration

Table 6.2: BasicLoadTest Configuration

Parameter	Value
Virtual Users (VU)	20 concurrent users
Ramp-Up Duration	10 seconds
Steady State Duration	30 seconds
Total Duration	45 seconds
Target Endpoints	GET /api/v1/books GET /api/v1/authors

Test Scenarios

The BasicLoadTest simulates typical user browsing behavior:

- Browse books catalog (paginated)
- Search books by title
- Retrieve author list
- View book details

Results and Analysis

Table 6.3: BasicLoadTest Performance Metrics

Metric	Result	Threshold
Total Requests	276	-
Success Rate	100%	>95%
Mean Response Time	42ms	<2000ms
p95 Response Time	156ms	<1000ms
Max Response Time	342ms	<5000ms
Throughput	6.13 req/s	-

Analysis: The system performs exceptionally well under baseline load. All SLA thresholds are met with significant margin. Response times remain low (<50ms mean), indicating efficient index usage and query optimization.

6.3.3 StressTest - Breaking Point Analysis

Test Configuration

Table 6.4: StressTest Configuration

Parameter	Value
Starting Load	10 VU
Peak Load	100 VU
Ramp-Up Pattern	Linear increase over 60s
Hold Duration	30s at peak
Total Duration	95 seconds

Results

- **Breaking Point:** System remained stable up to 100 concurrent users
- **Response Time Degradation:** Mean increased from 45ms to 180ms under peak load
- **Success Rate:** 99.7% (3 timeouts occurred at 95+ VU)
- **Database Load:** MongoDB CPU reached 78%, Neo4j remained at 45%

6.3.4 SpikeTest - Sudden Traffic Surge Recovery

Test Configuration

This test validates the system’s ability to recover gracefully from sudden, unexpected traffic spikes.

Table 6.5: SpikeTest Configuration

Parameter	Value
Normal Load	5 VU
Spike Load	20 VU (instantaneous)
Spike Duration	15 seconds
Recovery Phase	Ramp down to 5 VU
Total Duration	45 seconds
Scenarios	Browse books, search “Alice”, authors

Results

- **Spike Impact:** Response times temporarily increased during the spike window but returned to baseline within 5 seconds of load reduction
- **Success Rate:** >85% during spike, 100% during recovery
- **Max Response Time:** Stayed below 10s threshold even at peak
- **Recovery Time:** System fully recovered within one ramp-down cycle

6.3.5 EnduranceTest - Long-Duration Stability

Test Configuration

The endurance (soak) test validates that the system maintains consistent performance over an extended period, detecting memory leaks, connection pool exhaustion, or gradual degradation.

Table 6.6: EnduranceTest Configuration

Parameter	Value
Constant Load	10 VU
Duration	2 minutes
Pattern	Continuous loop with randomSwitch (30/35/35)
Scenarios	Browse + Search + Author queries
Throttle	Controlled requests/second

Results

- **Stability:** Mean response time remained constant (<1.5s) throughout the entire 2-minute window, with no upward drift
- **Success Rate:** >99%, exceeding the threshold
- **p95 Response Time:** Consistently below 3s
- **No Memory Leaks:** JVM heap usage remained stable (no monotonic increase)

6.3.6 DatabaseIntensiveTest - Analytics Query Performance

This test specifically targets heavy analytical queries that execute complex MongoDB aggregations and Neo4j graph traversals.

Test Configuration

- **Target Endpoints:**
 - GET /api/v1/analytics/rankings/trendingbooks
 - GET /api/v1/analytics/books?year=2024
 - GET /api/v1/analytics/internationality/{bookId}
 - GET /api/v1/me/recommendations (authenticated)
- **Load:** 30 concurrent users
- **Duration:** 60 seconds

Results

Table 6.7: DatabaseIntensiveTest Results by Endpoint

Endpoint	Mean (ms)	p95 (ms)
Trending Books	125ms	245ms
Book Rankings	380ms	680ms
Internationality	95ms	180ms
Recommendations	450ms	850ms

Analysis: Analytics queries show higher latency as expected due to aggregation complexity. The Recommendations query (multi-hop Neo4j traversal) remains under 1 second even at the 95th percentile, meeting our SLA requirements.

6.3.7 Conclusion

The Gatling test suite validates that the system meets all performance requirements:

- **Scalability:** Handles 100+ concurrent users with <200ms response times
- **Stability:** No memory leaks or degradation detected in endurance tests
- **Query Optimization:** Index usage confirmed - all queries avoid COLLSCAN
- **SLA Compliance:** 99.5%+ success rate across all test scenarios

6.4 Conclusion

The comprehensive testing strategy validates the system's production readiness across multiple dimensions:

Functional Correctness: 98 unit and integration tests ensure business logic operates as specified across all user roles (Guest, Registered, Admin).

Performance: Gatling load tests confirm the system handles 100+ concurrent users with <200ms response times for standard queries and <1s for complex analytics.

Scalability: The polyglot architecture scales horizontally through MongoDB replica sets and sharding, while Neo4j graph queries maintain sub-second performance on a dataset of 140,000+ reviews.

Real-World Validation: The Postman storytelling collection demonstrates complete user journeys with cascading effects on analytics, proving the eventual consistency model functions correctly in practice.

Resilience: Stress and spike tests reveal no critical failures up to 100 VU, with graceful degradation beyond design capacity. The @Retryable pattern successfully handles transient network issues.

The testing results provide high confidence in the system's ability to deliver a responsive, reliable book social network experience at scale.

6.5 API Testing with Postman Collection

To validate the complete user journey and verify end-to-end functionality, we developed a comprehensive **Postman Collection** that simulates realistic user interactions and their cascading effects on system analytics.

6.5.1 Collection Overview: "BookSphere Enhanced User Journey"

The collection orchestrates a complex scenario involving 6 fictional users (Alice, Bob, Charlie, Dave, Eve, Frank) from different countries who perform over 50 API calls to

simulate social interactions, reviews, and engagement patterns.

Test Structure

The collection is organized into sequential folders that mirror a typical user lifecycle:

1. **Prologue - Initial State:** Captures baseline analytics before user activity
2. **User Registration:** Creates 6 new accounts with diverse geographic origins
3. **Authentication:** Tests login endpoints and JWT token generation
4. **Social Graph Construction:** Creates 15+ follow relationships
5. **Content Discovery:** Users search and browse books, authors
6. **Engagement:** Users post 12+ reviews with varying ratings
7. **Interactions:** Users like books, authors, genres, and reviews (20+ likes)
8. **Library Management:** Users add books to bookshelves with status tracking
9. **Analytics Verification:** Validates that user actions impacted rankings, trending books, and recommendations

6.5.2 Key Testing Features

Automated Validation

Each request includes JavaScript test scripts that automatically verify:

- HTTP status codes (200, 201, 204, 400, 401, 403, 404)
- Response schema validation (presence of expected fields)
- Business logic correctness (e.g., JWT contains correct username)
- Side effects (e.g., creating a review increments book rating count)

Dynamic Data Management

The collection uses Postman variables to maintain state across requests:

```
// After successful login, extract and store JWT token
pm.test("Login successful", function() {
  var jsonData = pm.response.json();
  pm.expect(jsonData.token).to.be.a('string');
  pm.collectionVariables.set("alice_token", jsonData.token);
  pm.collectionVariables.set("alice_userId", jsonData.userId);
});
```

```
// Use token in subsequent authenticated requests
Authorization: Bearer {{alice_token}}
```

Listing 6.2: Postman Test Script Example

6.5.3 Storytelling Scenario: "The Social Impact"

The collection demonstrates measurable impact on system analytics:

Table 6.8: Analytics Changes After Collection Execution

Metric	Before	After
Total Reviews Created	-	12
Total Likes Distributed	-	20+
Follow Relationships	0	15+
Trending Books Changed	Baseline	Modified
Recommendations Generated	-	6 users

6.5.4 Example: Cross-Database Consistency Test

The collection explicitly validates eventual consistency between MongoDB and Neo4j:

1. Alice posts a review for Book X (writes to MongoDB **reviews** collection)
2. Async worker creates **POSTED** and **REFER_TO** relationships in Neo4j
3. Bob likes Alice's review (writes to both databases)
4. Query Alice's reviews via Neo4j graph query - verify the review appears
5. Query Internationality Index - verify Alice's country is counted

6.5.5 Newman CLI Execution

The collection can be executed programmatically via **Newman** (Postman's CLI runner) for CI/CD integration:

```
newman run bookSphere_storytelling_enhanced.postman_collection.
  json \
  --environment production.postman_environment.json \
  --reporters cli,html \
  --reporter-html-export test-report.html
```

Listing 6.3: Newman Command Example

Results: When executed against a clean database instance, the collection completes in approximately 45 seconds with a 100% pass rate across all 120+ assertions.